

Cloud Computing

Semester 2026-III

Delivery No. 1 — Requirements & Workload Model

Eng. Carlos Andrés Sierra, M.Sc.

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the first delivery of the *Cloud Computing* course project!

This delivery marks the starting point of a semester-long journey in which your team will design, architect, and validate a cloud-native distributed system for a real-world, high-traffic application. Before writing a single line of code or selecting any cloud service, you must deeply understand the problem your system needs to solve and the conditions under which it must operate.

The central question for this delivery is: **What does the system need to do, and under what conditions must it operate?**

Your answers to this question will drive every architecture decision made in the subsequent three deliveries. A well-defined requirements document is the foundation of a well-designed system.

General Delivery Definition: Requirements engineering and workload modeling involve translating a business problem into a precise, measurable specification of system behavior and operational constraints. In this delivery, teams will analyze their assigned application, identify all relevant actors and use cases, define functional and non-functional requirements, and produce a formal workload model that quantifies the expected system load. No cloud provider or technology selection is required at this stage.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavir-guezs@udistrital.edu.co*.

Delivery Scope and Objectives

- **Business Problem Understanding:** Deeply analyze your assigned application to understand its business model, value proposition, and the operational challenges that make it an interesting distributed systems problem.
- **Actor and Use Case Identification:** Identify all human and system actors that interact with the application and define the main use cases that the system must support.
- **Functional Requirements Specification:** Produce a catalog of 8–12 well-defined functional requirements that describe what the system must do.
- **Non-Functional Requirements Specification:** Define precise, measurable non-functional requirements covering availability, performance, scalability, data volume, and geographic scope.
- **Workload Model:** Produce a quantitative workload model that characterizes the expected system load under normal and peak conditions.
- **System Boundary Definition:** Clearly define what is inside and outside the scope of your system.

Required Artifacts and Methodology

1. System Description:

- *Business Problem:* Describe the real-world problem that your assigned application solves. What pain point does it address? Who benefits from it? What makes it operationally challenging at scale?
- *Actors:* Identify all actors (human users and external systems) that interact with your application. For each actor, describe their role, their interaction frequency, and their expectations regarding the system.
- *Main Use Cases:* Define the primary use cases of the system. For each use case, include: actor(s) involved, preconditions, main flow, alternative flows, and postconditions.
- *System Boundaries:* Define precisely what is inside the scope of your system and what is explicitly outside it. Use a system boundary diagram to visualize the scope.

2. Functional Requirements:

- Define between 8 and 12 functional requirements that describe what the system must do
- Each requirement must follow the format: “*The system shall [action] [object] [condition/constraint]*”

- Requirements must be: complete (no ambiguity), consistent (no contradictions), verifiable (can be tested), and traceable (linked to a use case)
- Organize requirements by functional area or service domain (e.g., user management, order processing, payment, notifications)
- Assign a unique identifier to each requirement (e.g., FR-01, FR-02) for traceability in future deliveries

3. Non-Functional Requirements:

Define precise, measurable non-functional requirements for each of the following categories:

- *Expected Users*: Total registered users and concurrent active users under normal and peak conditions
- *Average Requests/Hour*: Expected number of requests per hour under normal operating conditions
- *Peak Requests/Hour*: Maximum expected requests per hour during peak periods (e.g., dinner rush for DoorDash, Friday night for Netflix)
- *Peak Duration*: How long peak load periods typically last
- *Response-Time Targets*: Percentile-based response time targets (e.g., 95th percentile < 500 ms, 99th percentile < 2 s)
- *Availability*: Required uptime expressed as a percentage (e.g., 99.9%, 99.99%) and the corresponding maximum downtime per month
- *Data Volume*: Estimated total data volume at launch and projected growth per year
- *Data Growth*: Annual data growth rate in TB/year
- *Geographic Scope*: Regions or countries the system must serve, and any latency implications

Assign a unique identifier to each non-functional requirement (e.g., NFR-01, NFR-02).

4. Workload Model:

Produce a structured workload model table that quantifies the operational profile of your system. The workload model must include at minimum:

- Average requests/hour (normal load)
- Peak requests/hour
- Peak duration
- Read/write ratio
- Availability target (SLA)
- Response time target (percentile-based)
- Estimated data volume at launch

- Annual data growth
- Number of concurrent users (average and peak)
- Geographic distribution of traffic

All values must be justified with reference to publicly available data about your assigned application (engineering blogs, case studies, technical talks, or academic papers). Do not invent numbers without justification.

Example workload model format:

Parameter	Value
Average requests/hour	5,000
Peak requests/hour	30,000
Peak duration	45 minutes
Read/write ratio	80% / 20%
Availability	99.9%
Response time (p95)	< 500 ms
Data volume (launch)	5 TB
Data growth	2 TB/year
Concurrent users (peak)	50,000
Geographic scope	North America, Latin America

Deliverables and Documentation

1. Requirements & Workload Specification Report (PDF):

- *Executive Summary*: High-level description of the assigned application and the key challenges that make it a complex distributed systems problem
- *System Description*: Business problem, actor catalog, use case definitions, and system boundary diagram
- *Functional Requirements*: Numbered catalog of 8–12 functional requirements organized by domain
- *Non-Functional Requirements*: Numbered catalog of measurable non-functional requirements per category
- *Workload Model*: Structured table with all workload parameters and justification for each value
- *References*: Citations for all publicly available data used to justify workload values

2. GitHub Repository Setup:

- Create the course GitHub repository with the following base structure:
 - `delivery_1/` — requirements specification report and supporting diagrams
 - `delivery_2/` — software architecture document (future)

- `delivery_3/` — cloud architecture document (future)
 - `delivery_4/` — prototype, resilience report, and final architecture (future)
 - `prototype/` — AWS Academy vertical slice prototype (future)
- Provide a `README.md` documenting the team, assigned application, and repository structure

Submission Requirements

- The PDF report file must follow the naming convention: `groupN_delivery1.pdf`, where N is the team number (e.g., `group1_delivery1.pdf`, `group3_delivery1.pdf`)
- Submit the complete Requirements & Workload Specification Report as a single PDF through the designated course platform
- Include the **GitHub** repository link with the initial structure and Delivery 1 report
- All documentation must be in **English** and follow professional technical writing standards
- All workload values must cite their sources — requirements without justification will not be accepted
- **Do not select any cloud provider, cloud service, or technology stack in this delivery** — this is a requirements document, not a design document

Important Guidelines and Considerations

- **Research Your Application:** The best requirements documents are grounded in real engineering knowledge about the assigned application. Read engineering blogs (e.g., Uber Engineering, Netflix Tech Blog, DoorDash Engineering), watch technical talks, and study case studies to understand the real operational challenges.
- **Precision Over Completeness:** A requirement that says “the system shall be fast” is useless. A requirement that says “the system shall respond to ride-matching requests within 500 ms at the 95th percentile” is actionable. Prioritize precision.
- **Workload Numbers Must Be Justified:** Every number in the workload model must come from a source. Acceptable sources include: official engineering blogs, academic papers, public case studies, or credible news articles with usage statistics.
- **Think at Scale:** Your assigned application operates at massive scale. Think about millions of users, thousands of requests per second, and petabytes of data. Your workload model should reflect the real operational profile of the application.
- **Foundation for All Future Deliveries:** The requirements and workload model you define here will be the foundation for your software architecture (Delivery 2), cloud architecture (Delivery 3), and resilience analysis (Delivery 4). Invest time in getting this right.
- **No Technology Decisions:** This delivery is entirely technology-agnostic. Do not mention AWS, GCP, Azure, Kubernetes, or any specific database technology. Focus exclusively on *what* the system must do and *how* it must perform.

Good luck, and start your cloud architecture journey the right way — by deeply understanding the problem before thinking about the solution!